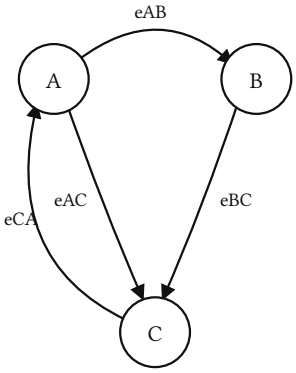
数据结构与算法
图

笔记整理自 *2026* 年 *10* 月 *28* 日课程

1. 十字链表

十字链表（orthogonal list）将有向图的每条弧同时挂入两条链：按弧尾串起的出弧链，以及按弧头串起的入弧链。因此，同一个弧结点需要保存两个后继指针。

有向图



十字链表

顶点结点

data	firstin	firstout
A	eCA	eAB
B	eAB	eBC
C	eAC	eCA

弧结点

弧	tailvex	headvex	hlink	tlink
eAB	A	B	null	eAC
eAC	A	C	eBC	null
eBC	B	C	null	null
eCA	C	A	null	null

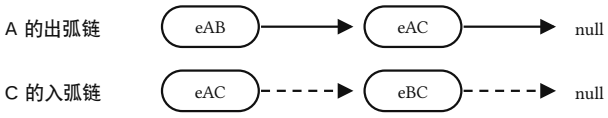
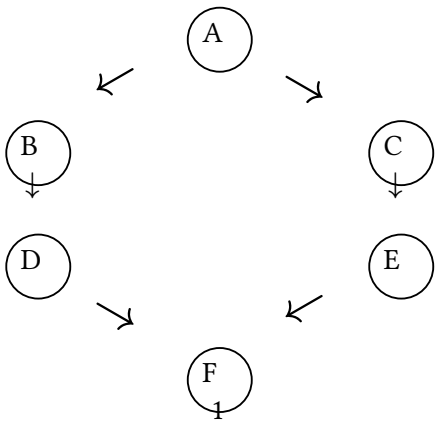


图 1：有向图及其十字链表。实线表示出弧链，虚线表示入弧链。

图中 `firstout` 指向以该顶点为弧尾的第一条弧，后续沿 `tlink` 查找；`firstin` 指向以该顶点为弧头的第一条弧，后续沿 `hlink` 查找。例如，顶点 A 的出弧链为 $e_{AB} \to e_{AC}$，顶点 C 的入弧链为 $e_{AC} \to e_{BC}$。

2. 拓扑排序

将一个有向无环图（directed acyclic graph, DAG）中的所有顶点排成一个线性序列，使每条边 $u \to v$ 的起点 u 都出现在终点 v 之前，这个序列称为**拓扑序**。若图中存在有向环，环上每个顶点都必须等待前驱，因而不存在拓扑序。



上图中，边 $A \to B$ 表示 A 必须排在 B 之前，其余边同理。序列 A, B, C, D, E, F 满足所有边的先后关系，因此是一个拓扑序。

2.1. 邻接矩阵

用邻接矩阵 M 表示上图：行表示边的起点，列表示边的终点。若存在边 $u \to v$，则 $M_{u,v} = 1$，否则为 0。

	A	**B**	**C**	**D**	**E**	**F**
A	0	1	1	0	0	0
B	0	0	0	1	0	0
C	0	0	0	0	1	0
D	0	0	0	0	0	1
E	0	0	0	0	0	1
F	0	0	0	0	0	0

矩阵中第 v 列的和就是顶点 v 的入度。因此，A 到 F 的初始入度依次为 $0, 1, 1, 1, 1, 2$。

2.2. BFS TopSort：用队列管理零入度顶点

可以把每个顶点看成一项任务，入边就是它尚未完成的前置任务。**入度就是还有多少个前置任务没完成**；入度为 0 时，该任务已经“解锁”，可以进入队列等待处理。

取出队头 $\to$ 加入拓扑序 $\to$ 删去它的出边 $\to$ 新的零入度点入队

算法先计算所有顶点的入度，并将所有零入度顶点入队。此后每次取出队头顶点 u，将 u 加入拓扑序，再遍历 u 的所有出边。每删去一条 $u \to v$，就将 v 的入度减 1；若 v 的入度变为 0，就将 v 加入队尾。

对上述 DAG 按字母顺序将零入度顶点入队，队列的变化如下：

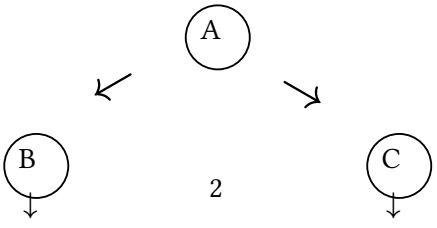
步骤	**取出**	**队列（队头在左）**	**当前拓扑序**
0	—	A	$\varnothing$
1	A	B、C	A
2	B	C、D	A, B
3	C	D、E	A, B, C
4	D	E	A, B, C, D
5	E	F	A, B, C, D, E
6	F	—	A, B, C, D, E, F

最终得到拓扑序 A, B, C, D, E, F。它之所以像 BFS，是因为队列会按“解锁”的先后顺序一层层处理顶点；与普通 BFS 不同的是，顶点入队的条件不是“尚未访问”，而是“入度刚好变为 0”。

如果同时有多个零入度顶点，它们的入队顺序可以不同，因此拓扑序不一定唯一。**算法结束后，若加入拓扑序的顶点数少于图中顶点总数，则图中存在有向环**。

2.3. DFS TopSort：回溯时压栈

BFS TopSort 从“已经没有前置任务”的顶点出发；DFS TopSort 则先沿出边不断向深处访问。它在刚访问顶点时不立即输出，而是等该顶点的所有后继都处理完成后，才在回溯时将它压入结果栈。



假设从 A 开始，并且先访问 B 这条分支。DFS 先深入 $A \to B \to D$；D 没有后继，因此先压入 D，然后回溯并依次压入 B。随后访问 $A \to C \to E$，依次压入 E、C，最后压入 A。因此压栈顺序为 D, B, E, C, A。

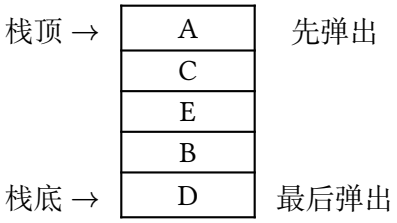


图 4：DFS 结束后的结果栈。

从栈顶依次弹出顶点，得到拓扑序 A, C, E, B, D。关键在于：对于每条边 $u \to v$，v 会先完成并压栈，u 后压栈却先弹出，因此最终 u 一定排在 v 之前。

DFS 会把顶点分为“未访问”、“正在访问”和“已完成”三种状态。如果 DFS 沿某条边到达一个“正在访问”的顶点，说明这条边回到了当前递归路径，图中存在有向环，不能进行拓扑排序。

对比	BFS TopSort	DFS TopSort
主要结构	入度数组和队列	访问状态、递归栈和结果栈
何时记录顶点	从队头取出时	所有后继完成、回溯时
如何发现环	输出顶点数不足	到达“正在访问”的顶点

增加一个 visiting 的状态维护可以判断是否存在环。

3. 带权图的最短路径问题

设带权图为 $G = (V, E, w)$，其中 V 是顶点集，E 是边集，权重函数 $w : E \to \mathbb{R}$ 为每条边指定一个实数权重。在有向图中，边 (u, v) 只允许从 u 走向 v；在无向图中，同一条边可以向两个方向通行。

从 u 到 v 的一条路径记为 $P = (v_0, v_1, ..., v_k)$，其中 $v_0 = u$、$v_k = v$，且每对相邻顶点之间都有边。路径的长度是路径上所有边权之和：

$$\ell(P) = \sum_{i=0}^{k-1} w(v_i, v_{i+1}).$$

最短路径是所有从 u 到 v 的路径中长度最小的一条，该最小长度称为最短距离，记为 $\delta(u, v)$。问题的输出通常既包括最短距离，也包括一条达到该距离的具体路径。

- 当 $u = v$ 时，空路径的长度为 0，因此在没有可达负权环影响时，$\delta(u, u) = 0$。
- 当从 u 无法到达 v 时，约定 $\delta(u, v) = \infty$。
- 如果从 u 可以到达某个负权环，并能再从该环到达 v，则可以反复绕环使路径长度不断减小，因而 u 到 v 不存在有限的最短距离。

3.1. 单源最短路径

给定一个源点 $s \in V$，求 s 到所有顶点 $v \in V$ 的最短距离 $\delta(s, v)$，并在需要时恢复对应的最短路径。若只给定一个终点 t，则称为**单对最短路径**，只需求 $\delta(s, t)$ 及一条从 s 到 t 的最短路径。

输入： $G = (V, E, w)$ 和源点 s。

输出： 对每个 $v \in V$，给出 $\delta(s, v)$；若要恢复路径，还需保存每个顶点在最短路径上的前驱。

Dijkstra 算法的实现策略。 以下讨论要求所有边权非负。

- 把所有顶点分成两组：
  - **已确定组**：从源点到这些顶点的最短距离已经“拍板”，以后不再修改。
  - **候选组**：只知道目前找到的最好路线，距离还可能继续缩短。
- 给每个候选顶点挂一块“距离路牌”，记录当前已知的最短距离。每轮选出路牌最小的顶点，把它从候选组移入已确定组。

初始化。

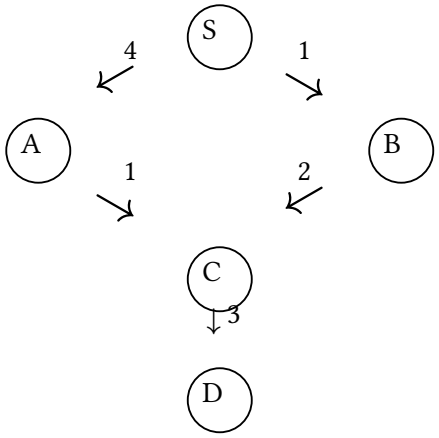
- 已确定组为空。
- 源点 s 的距离路牌写 0，其他顶点都写 ∞，表示暂时还没有找到可达路线。
- 每个顶点的前驱都留空；之后每次找到更短路线时，同时记下它是从哪个顶点走来的。

每轮操作。

1) 在候选组中找到距离路牌最小的顶点 u，将 u 移入已确定组。
2) 把 u 当作新的中转站，逐个检查它的邻居。如果“先到 u，再从 u 走过去”比邻居路牌上的距离更短，就改写路牌，并将该邻居的前驱记为 u。这个过程称为**松弛**。
3) 重复上述过程，直到所有可达顶点都已确定。如果只需要源点到终点 t 的路径，那么 t 被移入已确定组时就可以结束。

为什么最小路牌可以直接“拍板”？因为所有边权都非负，从其他尚未确定的顶点绕路只会让距离不变或变大，不可能在以后反过来得到一条更短的路。

例子。 下图中从 S 出发，线边的数字表示边权。



拍板顶点	已确定组	候选路牌	本轮变化
初始	空	S=0，其余为 ∞	只知道起点
S	S	A=4，B=1	发现 A、B
B	S、B	A=4，C=3	经 B 到 C 需要 3
C	S、B、C	A=4，D=6	经 C 到 D 需要 6
A	S、B、C、A	D=6	经 A 到 C 需要 5，不如原来的 3
D	S、B、C、A、D	空	全部完成

最终，S 到 A、B、C、D 的最短距离分别是 4、1、3、6。前驱记录为 A←S、B←S、C←B、D←C。例如，从 D 沿前驱倒着走得到 D←C←B←S，反转后就是最短路径 S→B→C→D，总长度为 6。

限制。Dijkstra 算法不适用于含负权边的图。负权边可能让一个已经 “拍板” 的距离在以后变得更小，从而破坏上述贪心选择的正确性。实现时，候选组可以直接扫描，也可以用最小堆快速取出距离最小的顶点。

3.2. 所有顶点对之间的最短路径

对每一对顶点 $u, v \in V$，求出最短距离 $\delta(u, v)$。输出可以表示为一个 $|V| \times |V|$ 的距离矩阵 D，其中 $D_{u,v} = \delta(u, v)$。若还需恢复具体路径，则需额外保存每对顶点之间的中间点或前驱信息。